Hybrid Deepfake Detection

SPATIAL-FREQUENCY ATTENTION NETWORK ARCHITECTURE

Architecture Overview

A deep learning framework for binary deepfake classification using a three-branch hybrid architecture. Combines spatial RGB features, channel-wise attention, and FFT frequency-domain analysis to detect manipulation artifacts across all FaceForensics++ forgery methods.

- **Spatial Branch:** Pretrained MobileNetV2 backbone extracts 1,280-dim RGB feature maps.
- **Attention Gate:** 1×1 Conv + Sigmoid re-weights feature channels toward manipulation regions.
- **Frequency Branch:** FFT magnitude spectrum (log-scaled) processed by 3-block CNN exposes GAN upsampling artifacts invisible in RGB space.
- **Classification Head:** Concatenated 1,344-dim vector → Dense(256) → Dense(64) → Dense(2) with Dropout.

Ablation Study Results

Baseline MobileNetV2	**88.59%**	Transfer learning reference
Attention CNN	**91.03%**	+2.44% — channel attention added
Hybrid Spatial-Frequency CNN	**93.34%**	+4.75% — selected production model

Dataset Specifications

Trained and evaluated on **FaceForensics++ C23** (pre-extracted faces). Seven manipulation classes collapsed to binary (Real=0, Fake=1). Balanced undersampling applied — 18,391 real vs 18,391 fake per training epoch.

Split	Images	Notes
Train	128,477	Balanced to 36,782 for training
Val	27,493	Full set used for validation
Test	26,276	Full set used for final evaluation

- **Kaggle Identifier:** gradientvoyager/faceforensics-c23-extracted-faces-100k
- **Archive Size:** ~2.46 GB

Technical Stack & Dependencies

- **Core Framework:** PyTorch, Torchvision

- **GPU Compute:** NVIDIA CUDA (validated on Tesla T4)
- **Evaluation:** Scikit-Learn — Classification Report, Confusion Matrix, AUC-ROC
- **Visualization:** Matplotlib, Seaborn, pytorch-grad-cam
- **Data Ingestion:** Kaggle API

```
pip install torch torchvision scikit-learn matplotlib seaborn numpy kaggle
grad-cam
```

Project Structure & Workflow

1. Data Ingestion & Extraction

```
import os

os.environ["KAGGLE_USERNAME"] = "your_kaggle_username"

os.environ["KAGGLE_KEY"] = "your_kaggle_api_key"

# Download and extract dataset

!kaggle datasets download -d
gradientvoyager/faceforensics-c23-extracted-faces-100k

!unzip -o -q faceforensics-c23-extracted-faces-100k.zip -d dataset/
```

2. Preprocessing & Augmentation

Train split applies RandomHorizontalFlip, RandomRotation(±10°), and ColorJitter before ImageNet normalization. Val/test splits apply normalization only. Pretrained MobileNetV2 requires ImageNet μ/σ rescaling.

3. Model Training

- Adam optimizer (lr=1e-4) with ReduceLROnPlateau scheduler (patience=1, factor=0.5)
- CrossEntropyLoss with label smoothing (0.1) to prevent overconfidence
- Per-epoch validation tracking with best-model checkpointing
- Training and validation learning curves saved as hybrid_training_curves.png

4. Model Serialization

```
# Best checkpoint (saved during training when val accuracy improves)

torch.save(model.state_dict(), "best_hybrid_model.pth")

# Final model after all epochs

torch.save(hybrid_model.state_dict(), "hybrid_deepfake_model.pth")
```

5. Performance Assessment

- Accuracy, Precision, Recall, F1-Score per class
- Confusion matrix (Real vs Fake)
- AUC-ROC curve with score — saved as roc_curve.png
- Grad-CAM visualizations showing spatial attention regions
- Ablation bar chart (val + test accuracy per model) — saved as ablation_study.png

Usage Instructions

1. **Workspace Setup:** Open the notebooks in a GPU-enabled environment (Google Colab with T4 recommended).
2. **Kaggle Credentials:** Set your username and API key in the credentials cell.
3. **Run Notebook 1** (DEEPLEARNING.ipynb) — Baseline MobileNetV2 and Attention CNN.
4. **Run Notebook 2** (Deepfake Research Extesion.ipynb) — Hybrid Spatial-Frequency model, full evaluation, and AUC-ROC.
5. **Output Files:** best_hybrid_model.pth, hybrid_deepfake_model.pth, roc_curve.png, ablation_study.png, hybrid_training_curves.png